MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Temciuc Adelina FAF-242

Report

on laboratory work no. 3
Cryptography and Security
Topic: Polyalphabetic Ciphers. The Vigenère Cipher

Checked by:
Zaica Maia, university assistant
DISA, FCIM, UTM

Chișinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to understand how polyalphabetic substitution ciphers hide the letter frequencies of a language, to study the Vigenère cipher as the classic example of such a cipher, and to implement it for the Romanian alphabet of 31 letters.

The laboratory work aims to:

- implement the Vigenère cipher (Task 3.2) for the Romanian alphabet, with the letters encoded by the numbers A = 0, Ă = 1, …, Z = 30;
- encrypt and decrypt strictly by the formulas of the mathematical model, $c = (m + k) \bmod 31$ and $m = (c - k) \bmod 31$;
- remove the spaces from the message and convert all letters to uppercase before processing;
- validate every input (the key, the message and the ciphertext), accept only Romanian letters, require a key of at least 7 letters and suggest the correct range of characters when the input is wrong;
- compare the security of the Vigenère cipher with that of the monoalphabetic Caesar cipher from laboratory work no. 1.

THEORETICAL BACKGROUND

Polyalphabetic substitution ciphers

The weakness of a monoalphabetic cipher, such as the Caesar cipher, is that every plaintext letter is always replaced by the same ciphertext letter. The frequency distribution of the ciphertext is therefore only a rearrangement of the distribution of the language, and frequency analysis breaks the cipher quickly.

A polyalphabetic cipher uses several substitution alphabets and changes the alphabet from one position of the text to the next. The same plaintext letter can then be encrypted into different letters, and different plaintext letters can be encrypted into the same letter. A frequent letter is mixed with rare ones, so the distribution of the ciphertext becomes flatter and gives much less information to the cryptanalyst. A simple example uses two alphabets, one for the even positions and one for the odd positions of the text, for instance $p_1(a) = 3a \bmod 26$ and $p_2(a) = (7a + 13) \bmod 26$.

The Vigenère cipher

The Vigenère cipher, first described by Giovan Battista Bellaso in 1553, uses the same operation as the Caesar cipher – a shift of the letter along the alphabet – but the shift is not the same for every letter. The key is a sequence of shifts $k = (k_1, k_2, \ldots, k_d)$, $0 \le k_i \le n - 1$, usually given as a word that is easy to remember: the code of every key letter is the shift. The key is written under the message and repeated as many times as needed. If m_i and c_i are the codes of the i-th plaintext and ciphertext letters and n is the length of the alphabet, then:

$$\textit{encryption: } c_i = (m_i + k_i) \bmod n, \qquad \textit{decryption: } m_i = (c_i - k_i) \bmod n,$$

where k_i is the code of the key letter found under position i, that is the key letter number i mod d. For the Romanian alphabet n = 31. The same operation can be done by hand with the Tabula Recta, a table whose rows are the alphabet shifted by 0, 1, …, n − 1 positions: the ciphertext letter is found at the intersection of the row of the key letter and the column of the message letter.

The cipher is much stronger than the Caesar cipher for two reasons: the length of the key is unknown to the attacker, and the number of possible keys grows exponentially with the key length. For a key of d letters there are 31^d keys; the minimum length of 7 letters required in this work already gives 31^7 = 27 512 614 111 keys, and a key of 10 letters gives $31^{10} \approx 8.2 \times 10^{14}$ keys.

The cipher is still not secure. Because the key repeats with period d, two equal fragments of the plaintext found at a distance that is a multiple of d are encrypted into equal fragments of the ciphertext. The Kasiski examination and the index of coincidence use this to find d; after that the ciphertext is split into d columns, each of which is an ordinary Caesar cipher that is broken by frequency analysis.

PART 1. IMPLEMENTATION

Task 3.2 was implemented in Python 3 without external libraries, in the same way as laboratory work no. 1. The program is a console application: the user chooses the operation, enters the key and the message or the ciphertext, and the result is printed in the terminal. The code is split into four modules:

- `alphabet.py` – the Romanian alphabet and the letter codes;
- `validation.py` – checking the key and the text;
- `cipher.py` – encryption and decryption;
- `main.py` – the menu and the dialogue with the user; the program is started with `python3 main.py`.

`main.py` imports the other three modules, `validation.py` and `cipher.py` use only `alphabet.py`, and the cipher functions know nothing about the console, so they can be tested and reused separately.

1.1 The letter encoding

In `alphabet.py` the alphabet is stored as a list in the order of Table 1.1, so the code of a letter is its index in the list. The dictionary `CODE` gives the code of every letter, and `TO_UPPER` maps every accepted character to its uppercase letter. The cedilla forms Ş and Ţ, which some keyboards produce, are mapped to the correct letters Ș and Ț.

Table 1.1 – Letter encoding of the Romanian alphabet (n = 31)

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
x	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
m	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	

```
# The Romanian alphabet (n = 31), letter codes A = 0 ... Z = 30.
ALPHABET = list("AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ")
LOWER = list("aăâbcdefghiîjklmnopqrsștțuvwxyz")
N = len(ALPHABET)
CODE = {letter: index for index, letter in enumerate(ALPHABET)}

# Every accepted character mapped to its uppercase letter. The cedilla forms
# Ş/ş and Ţ/ţ are treated as the comma-below letters Ș and Ț.
TO_UPPER = {}
for upper, lower in zip(ALPHABET, LOWER):
    TO_UPPER[upper] = upper
    TO_UPPER[lower] = upper
TO_UPPER.update({"Ş": "Ș", "ş": "Ș", "Ţ": "Ț", "ţ": "Ț"})

ALLOWED_LETTERS = "A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț"
```

No ASCII or Unicode code point is used in the computation: the codes come only from the position of the letter in the list, so the shift is always taken modulo 31 inside the Romanian alphabet.

1.2 Input validation

The module `validation.py` has one function for the text and one for the key. Each of them either returns a clean value or raises `InputError` with a message that shows the allowed characters. The text may contain only Romanian letters (in any case) and spaces. As the task requires, the spaces are removed first and then the letters are converted to uppercase; the first rejected character is reported together with its position:

```
def prepare_text(raw):
    """Check the characters, remove the spaces and convert to uppercase."""
    raw = unicodedata.normalize("NFC", raw)
    result = []
    for position, ch in enumerate(raw, start=1):
        if ch == " ":
            continue
        if ch not in TO_UPPER:
            raise InputError(
                f"Invalid character '{ch}' at position {position}.\n"
                f"Only Romanian letters ({ALLOWED_LETTERS}) and spaces "
                f"are allowed.")
        result.append(TO_UPPER[ch])
    if not result:
        raise InputError("The text is empty. Enter at least one letter.")
    return "".join(result)
```

The key follows the same rules, but spaces are not allowed inside it and it must have at least 7 letters:

```
def prepare_key(raw):
    """The key: only Romanian letters, at least 7, converted to uppercase."""
    raw = unicodedata.normalize("NFC", raw.strip())
    result = []
    for position, ch in enumerate(raw, start=1):
        if ch not in TO_UPPER:
            shown = "space" if ch == " " else f"'{ch}'"
            raise InputError(
                f"Invalid character {shown} at position {position}.\n"
                f"The key may contain only Romanian letters "
                f"({ALLOWED_LETTERS}).")
        result.append(TO_UPPER[ch])
    if len(result) < 7:
        raise InputError(f"The key has {len(result)} letters. "
                         f"It must have at least 7 letters.")
    return "".join(result)
```

The function `ask()` in `main.py` repeats the question until the input is accepted, printing the error each time, so the user never has to restart the program:

```
def ask(prompt, parser):
    """Ask again and again until the parser accepts the value."""
    while True:
        try:
            return parser(input(prompt))
        except InputError as error:
            print("  Error: " + str(error).replace("\n", "\n         "))
```

1.3 Encryption and decryption

Both operations are in `cipher.py` and share one function, `vigenere()`. For the letter at position i it takes the key letter at position i mod len(key), so the key is repeated along the text, adds or subtracts its code and reduces the result modulo 31. The parameter `sign` is 1 for encryption, which gives c = (m + k) mod 31, and −1 for decryption, which gives m = (c − k) mod 31. The operator % in Python always returns a value from 0 to 30, so a negative difference is brought back into the alphabet automatically:

```
from alphabet import ALPHABET, CODE, N

def vigenere(text, key, sign):
    """Shift letter i by the code of key letter i mod len(key)."""
    out = []
    for i, letter in enumerate(text):
        k = CODE[key[i % len(key)]]
        out.append(ALPHABET[(CODE[letter] + sign * k) % N])
    return "".join(out)


def encrypt(text, key):
    return vigenere(text, key, 1)       # c = (m + k) mod n


def decrypt(text, key):
    return vigenere(text, key, -1)      # m = (c - k) mod n
```

Table 1.2 shows the computation done by hand for the message „Știința criptografiei” and the key „securitate”, in the same way as the example from the laboratory materials. After the spaces are removed the message has 20 letters, so the 10-letter key is written twice.

Table 1.2 – Vigenère encryption with k = SECURITATE

m	Ș	T	I	I	N	Ț	A	C	R	I
k	S	E	C	U	R	I	T	A	T	E
m	22	23	10	10	16	24	0	4	20	10
k	21	6	4	25	20	10	23	0	23	6
m+k mod 31	12	29	14	4	5	3	23	4	12	16
c	J	Y	L	C	D	B	T	C	J	N
m	P	T	O	G	R	A	F	I	E	I
k	S	E	C	U	R	I	T	A	T	E
m	18	23	17	8	20	0	7	10	6	10
k	21	6	4	25	20	10	23	0	23	6
m+k mod 31	8	29	21	2	9	10	30	10	29	16
c	G	Y	S	Â	H	I	Z	I	Y	N

The result is JYLCDBTCJNGYSÂHIZIYN. For example, Ș (code 22) with the key letter S (code 21) gives (22 + 21) mod 31 = 12, which is J, and F (code 7) with T (code 23) gives 30, which is Z. Decryption subtracts the same codes: (12 − 21) mod 31 = −9 + 31 = 22, which is Ș again.

1.4 The command-line interface

The module `main.py` holds the dialogue with the user. The main menu offers encryption, decryption and an exit option. `run_task()` asks for the key and then for the message or the ciphertext, and prints the result. Before the result, `show_table()` prints the same table as Table 1.2 – the letters, the repeated key, their codes and the codes of the result – so that every letter can be checked by hand:

```
def show_table(text, key, result):
    repeated = "".join(key[i % len(key)] for i in range(len(text)))
    print("Text:  " + " ".join(f"{letter:>2}" for letter in text))
    print("Key:   " + " ".join(f"{letter:>2}" for letter in repeated))
    print("       " + " ".join(f"{CODE[letter]:>2}" for letter in text))
    print("       " + " ".join(f"{CODE[letter]:>2}" for letter in repeated))
    print("       " + " ".join(f"{CODE[letter]:>2}" for letter in result))
    print("Out:   " + " ".join(f"{letter:>2}" for letter in result))

def run_task(operation):
    key = ask("Key (at least 7 Romanian letters): ", prepare_key)
    if operation == "e":
        text = ask("Message: ", prepare_text)
        result = encrypt(text, key)
        show_table(text, key, result)
        print(f"Prepared message:  {text}")
        print(f"Ciphertext:        {result}")
    else:
        text = ask("Ciphertext: ", prepare_text)
        result = decrypt(text, key)
        show_table(text, key, result)
        print(f"Prepared text:     {text}")
        print(f"Decrypted message: {result}")
```

PART 2. RESULTS

2.1 Encryption

The message „Știința criptografiei” was encrypted with the key „securitate”. The program removed the space, converted the letters to uppercase, giving ȘTIINȚACRIPTOGRAFIEI, and printed the ciphertext JYLCDBTCJNGYSÂHIZIYN, which matches the computation from Table 1.2.

```
student@ubuntu:~/vigenere-lab3$ python3 main.py
=== Vigenère cipher - Romanian alphabet (n = 31) ===
Alphabet: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ

1. Encrypt
2. Decrypt
0. Exit
Choose: 1
Key (at least 7 Romanian letters): securitate
Message: Știința criptografiei
Text:   Ș  T  I  I  N  Ț  A  C  R  I  P  T  O  G  R  A  F  I  E  I
Key:    S  E  C  U  R  I  T  A  T  E  S  E  C  U  R  I  T  A  T  E
       22 23 10 10 16 24  0  4 20 10 18 23 17  8 20  0  7 10  6 10
       21  6  4 25 20 10 23  0 23  6 21  6  4 25 20 10 23  0 23  6
       12 29 14  4  5  3 23  4 12 16  8 29 21  2  9 10 30 10 29 16
Out:    J  Y  L  C  D  B  T  C  J  N  G  Y  S  Â  H  I  Z  I  Y  N
Prepared message:  ȘTIINȚACRIPTOGRAFIEI
Ciphertext:        JYLCDBTCJNGYSÂHIZIYN

1. Encrypt
2. Decrypt
0. Exit
Choose: 0
Bye!
student@ubuntu:~/vigenere-lab3$
```

Figure 2.1 – Encryption with the Vigenère cipher, k = securitate

The result shows the main property of a polyalphabetic cipher: the letter I appears five times in the message and is encrypted as L, C, N, I and N, while different letters can give the same ciphertext letter: Ș and R were both encrypted as J. It also shows its weakness: the letters at positions 2 and 12 (T and T) both became Y, and the letters at positions 10 and 20 (I and I) both became N, because they are exactly one key length (10 letters) apart. Repetitions like these are what the Kasiski examination looks for.

2.2 Decryption

The ciphertext was then decrypted. To check the case conversion, it was typed in lowercase and the key was typed as „Securitate”. The program subtracted the key codes and gave back the original message ȘTIINȚACRIPTOGRAFIEI. The final step of putting the spaces back, „ȘTIINȚA CRIPTOGRAFIEI”, is done by the reader.

```
student@ubuntu:~/vigenere-lab3$ python3 main.py
=== Vigenère cipher - Romanian alphabet (n = 31) ===
Alphabet: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ

1. Encrypt
2. Decrypt
0. Exit
Choose: 2
Key (at least 7 Romanian letters): Securitate
Ciphertext: jylcdbtcjngysâhiziyn
Text:   J  Y  L  C  D  B  T  C  J  N  G  Y  S  Â  H  I  Z  I  Y  N
Key:    S  E  C  U  R  I  T  A  T  E  S  E  C  U  R  I  T  A  T  E
       12 29 14  4  5  3 23  4 12 16  8 29 21  2  9 10 30 10 29 16
       21  6  4 25 20 10 23  0 23  6 21  6  4 25 20 10 23  0 23  6
       22 23 10 10 16 24  0  4 20 10 18 23 17  8 20  0  7 10  6 10
Out:    Ș  T  I  I  N  Ț  A  C  R  I  P  T  O  G  R  A  F  I  E  I
Prepared text:     JYLCDBTCJNGYSÂHIZIYN
Decrypted message: ȘTIINȚACRIPTOGRAFIEI

1. Encrypt
2. Decrypt
0. Exit
Choose: 0
Bye!
student@ubuntu:~/vigenere-lab3$
```

Figure 2.2 – Decryption with the Vigenère cipher, k = Securitate

2.3 Invalid input

Figure 2.3 shows how the program reacts to wrong input: the menu option 3 does not exist, the key „cheie” has only 5 letters, „parola123” contains digits and „cheie secretă” contains a space. After a correct key, the message „Lab nr. 3!” was rejected because of the dot, and a message made only of spaces was rejected as empty. Each error message states the allowed characters, and after each error the program asked again. Finally the message „Laboratorul trei” was encrypted.

```
student@ubuntu:~/vigenere-lab3$ python3 main.py
=== Vigenère cipher - Romanian alphabet (n = 31) ===
Alphabet: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ

1. Encrypt
2. Decrypt
0. Exit
Choose: 3
  Error: choose one of: 1, 2, 0.
Choose: 1
Key (at least 7 Romanian letters): cheie
  Error: The key has 5 letters. It must have at least 7 letters.
Key (at least 7 Romanian letters): parola123
  Error: Invalid character '1' at position 7.
         The key may contain only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț).
Key (at least 7 Romanian letters): cheie secretă
  Error: Invalid character space at position 6.
         The key may contain only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț).
Key (at least 7 Romanian letters): securitate
Message: Lab nr. 3!
  Error: Invalid character '.' at position 7.
         Only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț) and spaces are allowed.
Message:
  Error: The text is empty. Enter at least one letter.
Message: Laboratorul trei
Text:   L  A  B  O  R  A  T  O  R  U  L  T  R  E  I
Key:    S  E  C  U  R  I  T  A  T  E  S  E  C  U  R
       14  0  3 17 20  0 23 17 20 25 14 23 20  6 10
       21  6  4 25 20 10 23  0 23  6 21  6  4 25 20
        4  6  7 11  9 10 15 17 12  0  4 29 24  0 30
Out:    C  E  F  Î  H  I  M  O  J  A  C  Y  Ț  A  Z
Prepared message:  LABORATORULTREI
Ciphertext:        CEFÎHIMOJACYȚAZ

1. Encrypt
2. Decrypt
0. Exit
Choose: 0
Bye!
student@ubuntu:~/vigenere-lab3$
```

Figure 2.3 – Invalid key and text

CONCLUSION

In this laboratory work the Vigenère cipher was implemented in Python for the Romanian alphabet of 31 letters. The letters are encoded only by their position in the alphabet from Table 1.1, and the program follows the mathematical model exactly: $c_i = (m_i + k_i) \bmod 31$ for encryption and $m_i = (c_i - k_i) \bmod 31$ for decryption, with the key repeated along the message. The results printed by the program were checked by hand in Table 1.2.

Compared with the Caesar cipher from laboratory work no. 1, the Vigenère cipher is much stronger. The same letter is encrypted into different letters depending on its position, so the frequency distribution of the ciphertext is flattened and simple frequency analysis no longer works. The key space grows as 31^d with the key length d; with the required minimum of 7 letters there are already more than 2.7 × 10^10 keys, instead of 30 for the Caesar cipher.

However, the cipher is not secure by modern standards. The key is repeated periodically, and the experiment showed it directly: equal letters placed one key length apart were encrypted into equal letters. Using such repetitions, the length of the key can be found, after which the cipher splits into several Caesar ciphers, each broken by frequency analysis. The longer and less regular the key, the harder this attack becomes.

The program also validates all the input: it rejects keys shorter than 7 letters and any character outside the Romanian alphabet, shows the allowed range of characters and asks again, and it accepts both the comma-below and the cedilla forms of Ș and Ț.

REFERENCES

1. Laboratory Work No. 3. Polyalphabetic Ciphers. Cryptography and Security course materials, UTM, 2026.
2. Stallings W. Cryptography and Network Security: Principles and Practice. 8th ed. Pearson, 2020.
3. Singh S. The Code Book. Anchor Books, 2000.
4. Python 3 documentation. [online]. Available: https://docs.python.org/3/

The full source code is available in the GitHub repository:
https://github.com/Adelina1905/cs-2026